how to get neo4j uri

ChatGPT said:

ChatGPT

To get the **Neo4j URI**, you'll need to access your Neo4j database connection details. Here's how you can find or determine it based on your setup:

1. Neo4j Desktop

If you're using Neo4j Desktop:

- Open Neo4j Desktop and locate the database you want to connect to.
- Click on the **"Manage"** button for that database.
- Go to the **"Connection Details"** tab.
- The URI will be in the format:

arduino
CopyEdit
bolt://localhost:7687

or, for secure connections:
php
CopyEdit
bolt+s://<hostname>:<port>

- Replace <hostname> and <port> as necessary for your setup.

2. Neo4j Aura (Cloud-based Neo4j)

If you're using Neo4j Aura:

- Log in to your Neo4j Aura Console.
- Select your database.
- Under **Connection Details**, you'll find the URI in the form of:

arduino
CopyEdit
neo4j+s://<database-id>.databases.neo4j.io

3. Standalone Neo4j Server

If you've installed Neo4j on a server:

- The default URI is usually:

arduino
CopyEdit
bolt://<server-ip>:7687

or:
arduino
CopyEdit
http://<server-ip>:7474

- Replace <server-ip> with the server's actual IP address or hostname.

4. From the Configuration File

If you have access to the Neo4j configuration file:

1. Open the file located at:

javascript
CopyEdit
<neo4j-home>/conf/neo4j.conf

2. Look for the following settings:

makefile
CopyEdit
dbms.connector.bolt.listen_address=0.0.0.0:7687
dbms.connector.http.listen_address=0.0.0.0:7474

These indicate the URI and ports Neo4j is listening on.

Let me know if you'd like help configuring or troubleshooting this further!

4o

You said:

pip install

ChatGPT said:

ChatGPT

To install packages using pip, follow the steps below based on your situation:

Basic Syntax

The general syntax to install a Python package is:

bash
CopyEdit
pip install <package-name>

For example, to install **Neo4j Python Driver**, you can run:

bash
CopyEdit
pip install neo4j

Installing a Specific Version

If you want to install a specific version of a package:

bash
CopyEdit
pip install <package-name>==<version>

For example:

bash
CopyEdit

pip install neo4j==5.10.0

Installing from a Requirements File

If you have a list of dependencies in a requirements.txt file:

bash
CopyEdit
pip install -r requirements.txt

Upgrading a Package

To upgrade a package to the latest version:

bash
CopyEdit
pip install --upgrade <package-name>

Installing for a Specific User

To install a package only for the current user (no admin permissions required):

bash
CopyEdit
pip install --user <package-name>

Installing in a Virtual Environment

If you're working in a **virtual environment** (recommended for isolating dependencies):

1. Create a virtual environment:

bash
CopyEdit
python -m venv myenv

2. Activate it:
    a. **Windows**:

bash
CopyEdit
myenv\Scripts\activate

    b. **Mac/Linux**:

bash
CopyEdit
source myenv/bin/activate

3. Install your package:

bash
CopyEdit
pip install <package-name>

Common Issues

1. **Permission Denied**: Use --user or run with sudo (Linux/macOS):

bash
CopyEdit
sudo pip install <package-name>

2. **Pip Not Found**: Ensure pip is installed or updated:

bash
CopyEdit
python -m ensurepip --upgrade

3. **Proxy or Network Issues**: Use --proxy:

bash
CopyEdit
pip install <package-name> --proxy <proxy-address>

Let me know if you're encountering specific issues or need guidance with a particular library!

4o

You said:

in vs code WHAT INTEPRETER TO USE (base) alexfaith@Alexs-MacBook-Air st207 summative 3 % pip install neo4j Requirement already satisfied: neo4j in /opt/anaconda3/lib/python3.12/site-packages (5.27.0) Requirement already satisfied: pytz in /opt/anaconda3/lib/python3.12/site-packages (from neo4j) (2024.1) (base) alexfaith@Alexs-MacBook-Air st207 summative 3 % /Users/alexfaith/Downloads/st207 summative 3/.venv/bin/python -m pip install neo4j zsh: no such file or directory: /Users/alexfaith/Downloads/st207 (base) alexfaith@Alexs-MacBook-Air st207 summative 3 % pip install neo4j Requirement already satisfied: neo4j in /opt/anaconda3/lib/python3.12/site-packages (5.27.0) Requirement already satisfied: pytz in /opt/anaconda3/lib/python3.12/site-packages (from neo4j) (2024.1) (base) alexfaith@Alexs-MacBook-Air st207 summative 3 %

ChatGPT said:

ChatGPT

It seems you're running into issues with selecting the correct Python interpreter and potentially using a virtual environment for your project. Here's how to resolve it in VS Code:

1. Select the Correct Python Interpreter in VS Code

To ensure you're using the right interpreter:

1. Open **VS Code** and load your project folder (st207 summative 3).
2. Press Cmd + Shift + P (on Mac) or Ctrl + Shift + P (on Windows/Linux) to open the **Command Palette**.
3. Search for Python: Select Interpreter.
4. Choose the appropriate interpreter from the list:
    a. If you're using Anaconda, it might look like:

bash
CopyEdit

Python 3.12.0 ('base': conda) /opt/anaconda3/bin/python

b. If you're using a virtual environment, it should be:

bash
CopyEdit
Python 3.x.x (venv) /Users/alexfaith/Downloads/st207 summative 3/.venv/bin/python

If you don't see the desired interpreter, make sure it's installed and available.

2. Create or Fix Your Virtual Environment

If the virtual environment (.venv) is missing or broken:

1. Navigate to your project directory:

bash
CopyEdit
```
cd "/Users/alexfaith/Downloads/st207 summative 3"
```

2. Create a new virtual environment:

bash
CopyEdit
```
python3 -m venv .venv
```

3. Activate the virtual environment:
   a. On macOS/Linux:

bash
CopyEdit
```
source .venv/bin/activate
```

   b. On Windows:

bash
CopyEdit
```
.venv\Scripts\activate
```

4. Install Neo4j in the virtual environment:

bash
CopyEdit
```
pip install neo4j
```

3. Update VS Code Settings for the Virtual Environment

Once the virtual environment is created:

1. Select the virtual environment's Python interpreter in VS Code (see **Step 1**).
2. Ensure your settings.json file in VS Code includes:

```
json
CopyEdit
{
  "python.pythonPath": "/Users/alexfaith/Downloads/st207 summative
3/.venv/bin/python"
}
```

4. Troubleshooting the Path Issue

The error:

```
bash
CopyEdit
zsh: no such file or directory: /Users/alexfaith/Downloads/st207
```

is caused by the space in the directory name (st207 summative 3). When using paths with spaces in the terminal:

- Enclose the path in quotes:

```
bash
CopyEdit
"/Users/alexfaith/Downloads/st207 summative 3/.venv/bin/python" -m pip install neo4j
```

- Or escape the space with a backslash:

```
bash
CopyEdit
/Users/alexfaith/Downloads/st207\ summative\ 3/.venv/bin/python -m pip install neo4j
```

5. Verify Installation

After setting up:

1. Check which Python you're using:

bash
CopyEdit
which python

Ensure it points to your .venv.

2. Check installed packages:

bash
CopyEdit
pip list

Ensure neo4j is listed.

Let me know if you run into further issues!

4o

You said:

WHAT INPRETER SHOULD I USE I need neo 4j which is downloaded on. my compuetr

ChatGPT said:

ChatGPT

You should use the Python interpreter where **Neo4j** is installed. Based on your logs, Neo4j is already installed in the **Anaconda base environment**, located at:

bash
CopyEdit
/opt/anaconda3/lib/python3.12/site-packages

Thus, the best Python interpreter to use in VS Code is the **Anaconda base environment** interpreter. Here's how to ensure you're using the correct one:

Steps to Set the Correct Interpreter

1. **Open VS Code.**
2. **Select the Python Interpreter:**
    a. Press Cmd + Shift + P (on macOS) or Ctrl + Shift + P (on Windows/Linux) to open the Command Palette.
    b. Search for Python: Select Interpreter and click it.
    c. From the list, select the interpreter that matches:

bash
CopyEdit
Python 3.12.0 ('base': conda) /opt/anaconda3/bin/python

3. **Verify the Interpreter in VS Code Terminal:**
    a. Open the integrated terminal in VS Code (Ctrl + ~).
    b. Check the active interpreter by running:

bash
CopyEdit
which python

It should return:
bash
CopyEdit
/opt/anaconda3/bin/python

4. **Test Neo4j in Your Project:**
    a. Run the following command in the terminal to verify Neo4j is accessible:

python
CopyEdit
python -c "import neo4j; print(neo4j.__version__)"

    b. This should print the installed version of Neo4j, e.g., 5.27.0.

Optional: Virtual Environment (if isolation is needed)

If you prefer to use a **virtual environment** instead of the Anaconda base environment:

1. Create a virtual environment:

bash
CopyEdit

python -m venv .venv

2. Activate it:

bash
CopyEdit
source .venv/bin/activate

3. Install Neo4j in the virtual environment:

bash
CopyEdit
pip install neo4j

4. Switch the VS Code interpreter to:

bash
CopyEdit
/path/to/your/project/.venv/bin/python

When to Use Anaconda vs Virtual Environment

- **Anaconda Base Environment**: Use this if you're already working within Anaconda and don't need strict environment isolation.
- **Virtual Environment**: Use this if you want a clean, isolated environment specific to this project.

For your current setup, the Anaconda base environment seems like the easiest option since Neo4j is already installed there.

4o

You said:

with driver.session(database="neo4j") as session: for _, row in fixtures_df.iterrows(): query = """ MERGE (match:Match {id: $match_id}) SET match.date = $date, match.time = $time, match.home_team = $home_team, match.away_team = $away_team, match.home_expected = $home_expected, match.away_expected = $away_expected, match.home_score = $home_score, match.away_score = $away_score MERGE (ref:Referee {name: $referee}) MERGE (venue:Venue {name: $venue}) MERGE (home_team:Team {name: $home_team}) MERGE (away_team:Team {name: $away_team}) MERGE (match)-[:REFERRED_BY]->(ref) MERGE (match)-[:PLAYED_AT]->(venue) MERGE (match)-[:HOME_TEAM]->(home_team) MERGE (match)-

[:AWAY_TEAM]->(away_team) """ # Prepare the data dictionary data ={ "match_id": f"GW{row['GW']}_{row['Home']}_vs_{row['Away']}", "date": row["Date"], "time": row["Time"], "home_team": row["Home"], "away_team": row["Away"], } if row["Match Report"]: data["home_expected"]= row["xG"], data["away_expected"]= row["xG.1"], data["home_score"]= row["Home_Score"], data["away_score"]= row["Away_Score"], data["referee"]= row["Referee"], data["venue"] = row["Venue"] # Execute the query session.run(query, data) ile /opt/anaconda3/lib/python3.12/site-packages/neo4j/_sync/io/_common.py:254, in Response.on_failure(self, metadata) 252 handler = self.handlers.get("on_summary") 253 Util.callback(handler) --> 254 raise self._hydrate_error(metadata) ClientError: {code: Neo.ClientError.Statement.SemanticError} {message: Cannot merge the following node because of NaN property value for 'name': (:Team {name: NaN})}

ChatGPT said:

ChatGPT

The error indicates that a NaN value is being passed to the name property of the Team node in your Cypher query. Neo4j does not allow NaN values for properties, so you need to handle missing or invalid values in your fixtures_df DataFrame before executing the query.

Here's how you can fix it:

1. Check for Missing Values in fixtures_df

Before running the query, ensure that all required fields have valid values:

python
CopyEdit
```
print(fixtures_df.isna().sum())
```

This will show the count of missing values for each column. If Home or Away has missing values, you'll need to handle them.

2. Handle Missing or NaN Values

Update your code to replace missing values with default values or skip rows with missing data. For example:

Option 1: Replace Missing Values

Replace NaN with a default value (e.g., "Unknown"):

python
CopyEdit
```
fixtures_df.fillna({"Home": "Unknown", "Away": "Unknown"}, inplace=True)
```

Option 2: Skip Rows with Missing Values

Skip rows where critical columns like Home or Away are NaN:

python
CopyEdit
```
fixtures_df = fixtures_df.dropna(subset=["Home", "Away"])
```

3. Update Data Preparation

In your data dictionary, ensure no NaN values are included:

python
CopyEdit
```
data = {
   "match_id": f"GW{row['GW']}_{row['Home']}_vs_{row['Away']}",
   "date": row["Date"],
   "time": row["Time"],
   "home_team": row["Home"] or "Unknown",
   "away_team": row["Away"] or "Unknown",
}

if row["Match Report"]:
   data.update({
      "home_expected": row["xG"] if not pd.isna(row["xG"]) else 0,
      "away_expected": row["xG.1"] if not pd.isna(row["xG.1"]) else 0,
```

```
    "home_score": row["Home_Score"] if not pd.isna(row["Home_Score"]) else 0,
    "away_score": row["Away_Score"] if not pd.isna(row["Away_Score"]) else 0,
    "referee": row["Referee"] or "Unknown",
    "venue": row["Venue"] or "Unknown",
  })
```

This ensures no NaN or invalid values are passed to Neo4j.

4. Test and Debug

After handling missing values:

1. Test the updated DataFrame:

python
CopyEdit

```
print(fixtures_df.head())
```

Ensure no NaN values exist in columns that are required for the query.

2. Rerun your script to ensure the issue is resolved.

5. Optional Logging for Debugging

Add logging to identify problematic rows:

python
CopyEdit

```
try:
  session.run(query, data)
except Exception as e:
  print(f"Failed for match: {data['match_id']}")
  print(f"Error: {e}")
```

This will help you pinpoint rows causing issues in the future.

Let me know if you need further assistance!

4o

You said:

why do the missing values cause error

ChatGPT said:

ChatGPT

The missing values cause an error because Neo4j does not allow NaN (Not a Number) or None as property values when creating or merging nodes. Here's a breakdown of why this happens:

1. Neo4j and Property Constraints

- **Property Value Rules:** In Neo4j, properties on nodes and relationships must have valid values such as string, integer, float, boolean, or list. Neo4j does **not** allow:
  - NaN (from pandas DataFrame, typically from missing numerical values)
  - None (Python equivalent of null)
- When your query attempts to MERGE or SET a node with NaN as a property, Neo4j raises an error because NaN is not a valid value.

For example:

cypher
CopyEdit
```
MERGE (team:Team {name: NaN})
```

This will fail because NaN is not valid.

2. How Missing Values Lead to Errors

In your code:

- The fixtures_df DataFrame may have missing values (NaN) for columns such as Home, Away, or other fields.

- When you pass these NaN values into the data dictionary, they are sent to the Cypher query as invalid properties.

Example of what might happen:

python
CopyEdit
```
data = {
   "home_team": row["Home"],  # row["Home"] could be NaN
   "away_team": row["Away"],  # row["Away"] could be NaN
}
```

This causes the query:

cypher
CopyEdit
```
MERGE (home_team:Team {name: $home_team})
```

to fail because $home_team resolves to NaN.

3. Why MERGE Fails Specifically

The MERGE clause in Cypher is used to find or create a node with specific properties. If the property includes an invalid value (NaN), Neo4j:

1. Cannot resolve the node because the value is not valid.
2. Throws a ClientError (e.g., SemanticError) to indicate the invalid input.

4. Example Scenario

Assume a row in your fixtures_df DataFrame:

GW	Home	Away	Date	xG	xG.1	Home_Score	Away_Score	Referee	Venue
1	NaN	Arsenal	2023-08-01	NaN	1.5	0	2	Anthony T.	Emirates

This row has NaN for:

- Home (team name is missing)
- xG (expected goals for home team)

When this row is processed:

1. The data dictionary might include:

python
CopyEdit
```
{
  "home_team": NaN,
  "away_team": "Arsenal",
  "home_expected": NaN,
  "away_expected": 1.5,
}
```

2. This results in a query like:

cypher
CopyEdit
```
MERGE (home_team:Team {name: NaN})
```

which is invalid in Neo4j.

5. How to Fix This

You must preprocess the data to replace NaN or None with valid fallback values.

Key Fixes:

- Replace missing team names (NaN) with a default like "Unknown":

python
CopyEdit
```
fixtures_df.fillna({"Home": "Unknown", "Away": "Unknown"}, inplace=True)
```

- Skip rows where critical values are missing:

python
CopyEdit
```
fixtures_df = fixtures_df.dropna(subset=["Home", "Away"])
```

Ensure Valid Values in data Dictionary:

Use Python's or operator to provide fallback values for None or NaN:

python
CopyEdit
```
data = {
   "home_team": row["Home"] or "Unknown",
   "away_team": row["Away"] or "Unknown",
   "home_expected": row["xG"] if not pd.isna(row["xG"]) else 0,
   "away_expected": row["xG.1"] if not pd.isna(row["xG.1"]) else 0,
}
```

6. Testing the Fix

After applying these fixes:

1. Check for missing values in critical columns before running the query:

python
CopyEdit
```
print(fixtures_df.isna().sum())
```

2. Run the query, and the error should no longer occur.

Let me know if you'd like additional help!

4o

You said:

with driver.session(database="neo4j") as session: for i, row in fixtures_df[:len(match_reports)].iterrows(): match_id = f"GW{row['GW']}_{row['Home']}_vs_{row['Away']}" home_team_stats = match_reports[i][0][0] away_team_stats = match_reports[i][1][0] home_team_goaly_stats = match_reports[i][0][1] away_team_goaly_stats = match_reports[i][1][1] home_team_name = row['Home'] away_team_name = row['Away'] # Add Home Team Player Stats for _, player_stats in home_team_stats.iterrows(): player_query = """ MERGE (player:Player {name: $player_name, nation: $nation, position: $position, age: $age}) MERGE (team:Team {name: $team_name}) MERGE (player)-[:PLAYS_FOR]->(team) MERGE (match:Match

{id: $match_id}) MERGE (match)-[h:HOME_TEAM_PLAYER_STATS]->(player) SET h.goals = $goals, h.assists = $assists, h.penalties = $penalties, h.shots = $shots, h.shots_on_target = $shots_on_target, h.touches = $touches, h.tackles = $tackles, h.interceptions = $interceptions, h.blocks = $blocks, h.xg = $xg, h.npxg = $npxg, h.xag = $xag, h.sca = $sca, h.gca = $gca, h.passes_completed = $passes_completed, h.passes_attempted = $passes_attempted, h.pass_accuracy = $pass_accuracy, h.progressive_passes = $progressive_passes, h.carries = $carries, h.progressive_carries = $progressive_carries """ session.run(player_query, { "player_name": player_stats[('Unnamed: 0_level_0', 'Player')], "nation": player_stats[('Unnamed: 2_level_0', 'Nation')], "position": player_stats[('Unnamed: 3_level_0', 'Pos')], "age": player_stats[('Unnamed: 4_level_0', 'Age')], "match_id": match_id, "team_name": home_team_name, "goals": player_stats[('Performance', 'Gls')], "assists": player_stats[('Performance', 'Ast')], "penalties": player_stats[('Performance', 'PK')], "shots": player_stats[('Performance', 'Sh')], "shots_on_target": player_stats[('Performance', 'SoT')], "touches": player_stats[('Performance', 'Touches')], "tackles": player_stats[('Performance', 'Tkl')], "interceptions": player_stats[('Performance', 'Int')], "blocks": player_stats[('Performance', 'Blocks')], "xg": player_stats[('Expected', 'xG')], "npxg": player_stats[('Expected', 'npxG')], "xag": player_stats[('Expected', 'xAG')], "sca": player_stats[('SCA', 'SCA')], "gca": player_stats[('SCA', 'GCA')], "passes_completed": player_stats[('Passes', 'Cmp')], "passes_attempted": player_stats[('Passes', 'Att')], "pass_accuracy": player_stats[('Passes', 'Cmp%')], "progressive_passes": player_stats[('Passes', 'PrgP')], "carries": player_stats[('Carries', 'Carries')], "progressive_carries": player_stats[('Carries', 'PrgC')], }) # Add Away Team Player Stats for _, player_stats in away_team_stats.iterrows(): player_query = """ MERGE (player:Player {name: $player_name, nation: $nation, position: $position, age: $age}) MERGE (team:Team {name: $team_name}) MERGE (player)-[:PLAYS_FOR]->(team) MERGE (match:Match {id: $match_id}) MERGE (match)-[a:AWAY_TEAM_PLAYER_STATS]->(player) SET a.goals = $goals, a.assists = $assists, a.penalties = $penalties, a.shots = $shots, a.shots_on_target = $shots_on_target, a.touches = $touches, a.tackles = $tackles, a.interceptions = $interceptions, a.blocks = $blocks, a.xg = $xg, a.npxg = $npxg, a.xag = $xag, a.sca = $sca, a.gca = $gca, a.passes_completed = $passes_completed, a.passes_attempted = $passes_attempted, a.pass_accuracy = $pass_accuracy, a.progressive_passes = $progressive_passes, a.carries = $carries, a.progressive_carries = $progressive_carries """ session.run(player_query, { "player_name": player_stats[('Unnamed: 0_level_0', 'Player')], "nation": player_stats[('Unnamed: 2_level_0', 'Nation')], "position": player_stats[('Unnamed: 3_level_0', 'Pos')], "age": player_stats[('Unnamed: 4_level_0', 'Age')], "match_id": match_id, "team_name": away_team_name, "goals": player_stats[('Performance', 'Gls')], "assists": player_stats[('Performance', 'Ast')], "penalties": player_stats[('Performance', 'PK')], "shots": player_stats[('Performance',

'Sh')], "shots_on_target": player_stats[('Performance', 'SoT')], "touches": player_stats[('Performance', 'Touches')], "tackles": player_stats[('Performance', 'Tkl')], "interceptions": player_stats[('Performance', 'Int')], "blocks": player_stats[('Performance', 'Blocks')], "xg": player_stats[('Expected', 'xG')], "npxg": player_stats[('Expected', 'npxG')], "xag": player_stats[('Expected', 'xAG')], "sca": player_stats[('SCA', 'SCA')], "gca": player_stats[('SCA', 'GCA')], "passes_completed": player_stats[('Passes', 'Cmp')], "passes_attempted": player_stats[('Passes', 'Att')], "pass_accuracy": player_stats[('Passes', 'Cmp%')], "progressive_passes": player_stats[('Passes', 'PrgP')], "carries": player_stats[('Carries', 'Carries')], "progressive_carries": player_stats[('Carries', 'PrgC')], }) # Add Home Team Goalie Stats for _, goalie_stats in home_team_goaly_stats.iterrows(): goalie_query = """ MERGE (goalie:Player {name: $goalie_name, nation: $nation, position: "GK", age: $age}) MERGE (team:Team {name: $team_name}) MERGE (goalie)-[:PLAYS_FOR]->(team) MERGE (match:Match {id: $match_id}) MERGE (match)-[:HOME_TEAM_GOALIE_STATS]->(goalie) SET goalie.minutes_played = $minutes_played, goalie.sota = $sota, goalie.goals_allowed = $goals_allowed, goalie.saves = $saves, goalie.save_percentage = $save_percentage, goalie.psxg = $psxg, goalie.passes_completed = $passes_completed, goalie.passes_attempted = $passes_attempted, goalie.pass_accuracy = $pass_accuracy, goalie.gk_passes_attempted = $gk_passes_attempted, goalie.throws = $throws, goalie.launch_percentage = $launch_percentage, goalie.launch_average_length = $launch_average_length, goalie.goal_kicks_attempted = $goal_kicks_attempted, goalie.goal_kick_launch_percentage = $goal_kick_launch_percentage, goalie.goal_kick_average_length = $goal_kick_average_length, goalie.crosses_opportunities = $crosses_opportunities, goalie.crosses_stops = $crosses_stops, goalie.crosses_stop_percentage = $crosses_stop_percentage, goalie.opa = $opa, goalie.opa_average_distance = $opa_average_distance """ session.run(goalie_query, { "goalie_name": goalie_stats[('Unnamed: 0_level_0', 'Player')], "nation": goalie_stats[('Unnamed: 1_level_0', 'Nation')], "age": goalie_stats[('Unnamed: 2_level_0', 'Age')], "match_id": match_id, "team_name": home_team_name, "minutes_played": goalie_stats[('Unnamed: 3_level_0', 'Min')], "sota": goalie_stats[('Shot Stopping', 'SoTA')], "goals_allowed": goalie_stats[('Shot Stopping', 'GA')], "saves": goalie_stats[('Shot Stopping', 'Saves')], "save_percentage": goalie_stats[('Shot Stopping', 'Save%')], "psxg": goalie_stats[('Shot Stopping', 'PSxG')], "passes_completed": goalie_stats[('Launched', 'Cmp')], "passes_attempted": goalie_stats[('Launched', 'Att')], "pass_accuracy": goalie_stats[('Launched', 'Cmp%')], "gk_passes_attempted": goalie_stats[('Passes', 'Att (GK)')], "throws": goalie_stats[('Passes', 'Thr')], "launch_percentage": goalie_stats[('Passes', 'Launch%')], "launch_average_length": goalie_stats[('Passes', 'AvgLen')], "goal_kicks_attempted": goalie_stats[('Goal Kicks', 'Att')], "goal_kick_launch_percentage": goalie_stats[('Goal Kicks', 'Launch%')], "goal_kick_average_length": goalie_stats[('Goal Kicks', 'AvgLen')],

"crosses_opportunities": goalie_stats[('Crosses', 'Opp')], "crosses_stops": goalie_stats[('Crosses', 'Stp')], "crosses_stop_percentage": goalie_stats[('Crosses', 'Stp%')], "opa": goalie_stats[('Sweeper', '#OPA')], "opa_average_distance": goalie_stats[('Sweeper', 'AvgDist')], }) # Add Away Team Goalie Stats for _, goalie_stats in away_team_goaly_stats.iterrows(): session.run(goalie_query, { "goalie_name": goalie_stats[('Unnamed: 0_level_0', 'Player')], "nation": goalie_stats[('Unnamed: 1_level_0', 'Nation')], "age": goalie_stats[('Unnamed: 2_level_0', 'Age')], "match_id": match_id, "team_name": away_team_name, "minutes_played": goalie_stats[('Unnamed: 3_level_0', 'Min')], "sota": goalie_stats[('Shot Stopping', 'SoTA')], "goals_allowed": goalie_stats[('Shot Stopping', 'GA')], "saves": goalie_stats[('Shot Stopping', 'Saves')], "save_percentage": goalie_stats[('Shot Stopping', 'Save%')], "psxg": goalie_stats[('Shot Stopping', 'PSxG')], "passes_completed": goalie_stats[('Launched', 'Cmp')], "passes_attempted": goalie_stats[('Launched', 'Att')], "pass_accuracy": goalie_stats[('Launched', 'Cmp%')], "gk_passes_attempted": goalie_stats[('Passes', 'Att (GK)')], "throws": goalie_stats[('Passes', 'Thr')], "launch_percentage": goalie_stats[('Passes', 'Launch%')], "launch_average_length": goalie_stats[('Passes', 'AvgLen')], "goal_kicks_attempted": goalie_stats[('Goal Kicks', 'Att')], "goal_kick_launch_percentage": goalie_stats[('Goal Kicks', 'Launch%')], "goal_kick_average_length": goalie_stats[('Goal Kicks', 'AvgLen')], "crosses_opportunities": goalie_stats[('Crosses', 'Opp')], "crosses_stops": goalie_stats[('Crosses', 'Stp')], "crosses_stop_percentage": goalie_stats[('Crosses', 'Stp%')], "opa": goalie_stats[('Sweeper', '#OPA')], "opa_average_distance": goalie_stats[('Sweeper', 'AvgDist')], })File /opt/anaconda3/lib/python3.12/site-packages/neo4j/_sync/io/_common.py:254, in Response.on_failure(self, metadata) 252 handler = self.handlers.get("on_summary") 253 Util.callback(handler) --> 254 raise self._hydrate_error(metadata) ClientError: {code: Neo.ClientError.Statement.SemanticError}{message: Cannot merge the following node because of NaN property value for 'nation': (:Player {nation: NaN})}

ChatGPT said:

ChatGPT

The error occurs because the nation property of the Player node is NaN, which is not allowed in Neo4j. Neo4j does not support invalid values like NaN or None for node properties.

To fix this issue, you must sanitize the data before executing the Cypher queries. Specifically, replace NaN values with a valid default value (e.g., "Unknown" or an empty string) in your data processing pipeline.

Here’s how you can fix this:

1. Handle Missing Data in Python

Before passing data to the Cypher query, ensure there are no NaN or None values. You can achieve this using pandas to preprocess your data.

Example:

python
CopyEdit
```
import numpy as np

# Replace NaN values in player stats
home_team_stats.fillna({
   ('Unnamed: 2_level_0', 'Nation'): "Unknown",
   ('Unnamed: 3_level_0', 'Pos'): "Unknown",
   ('Unnamed: 4_level_0', 'Age'): 0,  # Replace missing age with 0
}, inplace=True)

away_team_stats.fillna({
   ('Unnamed: 2_level_0', 'Nation'): "Unknown",
   ('Unnamed: 3_level_0', 'Pos'): "Unknown",
   ('Unnamed: 4_level_0', 'Age'): 0,
}, inplace=True)

home_team_goaly_stats.fillna("Unknown", inplace=True)
away_team_goaly_stats.fillna("Unknown", inplace=True)
```

Alternatively, you can replace NaN values when creating the player_query parameters:

python
CopyEdit
```
def sanitize_value(value, default="Unknown"):
   return value if pd.notna(value) else default

session.run(player_query, {
   "player_name": sanitize_value(player_stats[('Unnamed: 0_level_0', 'Player')]),
   "nation": sanitize_value(player_stats[('Unnamed: 2_level_0', 'Nation')]),
```

```
    "position": sanitize_value(player_stats[('Unnamed: 3_level_0', 'Pos')]),
    "age": sanitize_value(player_stats[('Unnamed: 4_level_0', 'Age')], default=0),
    ...
})
```

2. Verify Data Before Running the Query

You can add

4o

You said:

what does this do with driver.session(database="neo4j") as session: # Create GameWeek nodes and relationships for gw in range(1, 39): # Loop from 1 to 38 query = """ MATCH (s:Season {name: $season_name}) CREATE (gw:GameWeek {number: $number})-[:IS_A_GW]->(s) """ session.run(query, {"season_name": "24-25", "number": gw})

ChatGPT said:

ChatGPT

The given code creates **GameWeek** nodes in the Neo4j database and establishes relationships between each GameWeek node and an existing Season node. Here's a detailed breakdown of its function:

1. Loop Through Game Weeks

python
CopyEdit
```
for gw in range(1, 39):
```

This iterates over game weeks (gw) from 1 to 38, representing a typical football league season with 38 game weeks.

2. Cypher Query

cypher
CopyEdit
```
MATCH (s:Season {name: $season_name})
CREATE (gw:GameWeek {number: $number})-[:IS_A_GW]->(s)
```

- **MATCH (s:Season {name: $season_name})**: Searches for a Season node with a property name equal to the value of season_name. This ensures that the GameWeek nodes are connected to the correct Season.
- **CREATE (gw:GameWeek {number: $number})**: Creates a GameWeek node with a property number equal to the value of gw (from the loop).
- **-[:IS_A_GW]->(s)**: Creates a directed relationship of type IS_A_GW from the GameWeek node to the matched Season node.

3. Session Execution

python
CopyEdit
```
session.run(query, {"season_name": "24-25", "number": gw})
```

For each iteration:

- season_name is set to "24-25", representing the name of the football season (2024-25).
- number is set to the current value of gw (ranging from 1 to 38).

This query creates:

- GameWeek nodes (GameWeek {number: 1} through GameWeek {number: 38}).
- Relationships of type IS_A_GW connecting these GameWeek nodes to the Season node with name: "24-25".

Example Neo4j Data Representation

After running this code, your Neo4j database will have:

1. A Season node:

plaintext
CopyEdit
(:Season {name: "24-25"})

2. 38 GameWeek nodes:

plaintext
CopyEdit
(:GameWeek {number: 1}), (:GameWeek {number: 2}), ..., (:GameWeek {number: 38})

3. Relationships:

plaintext
CopyEdit
(:GameWeek {number: 1})-[:IS_A_GW]->(:Season {name: "24-25"})
(:GameWeek {number: 2})-[:IS_A_GW]->(:Season {name: "24-25"})
...

Purpose

This code is organizing the season into its constituent game weeks in the graph database. This structure can be used for:

- Tracking matches played in each game week.
- Analyzing performance or statistics across game weeks.
- Structuring a season's timeline in Neo4j for queries and visualization.

4o

You said:

query to delete all existing nodes and relationships

ChatGPT said:

ChatGPT

To delete all existing nodes and relationships in a Neo4j database, you can use the following Cypher query:

cypher
CopyEdit
MATCH (n)
DETACH DELETE n

Explanation:

1. **MATCH (n)**:
    a. Matches all nodes in the database.
2. **DETACH DELETE n**:
    a. Deletes the nodes (n) and their relationships. The

4o

You said:

in python

ChatGPT said:

ChatGPT

In Python, you can run the query to delete all existing nodes and relationships using the Neo4j driver. Here's how:

python
CopyEdit
```
from neo4j import GraphDatabase

# Define your Neo4j connection details
uri = "bolt://localhost:7687"  # Adjust if necessary
username = "neo4j"
password = "your_password"

# Initialize the driver
driver = GraphDatabase.driver(uri, auth=(username, password))

# Delete all nodes and relationships
with driver.session(database="neo4j") as session:
    query = """
    MATCH (n)
```

```
    DETACH DELETE n
    """
    session.run(query)

print("All nodes and relationships have been deleted.")
driver.close()
```

Key Points:

- Replace "your_password" with the password for your Neo4j database.
- Ensure the database name ("neo4j") matches your target database.
- Always close the driver after use with driver.close() to release resources.

4o

You said:

Add Home Team Goalie Stats for _, goalie_stats in home_team_goaly_stats.iterrows(): goalie_query = """ MERGE (goalie:Player {name: $goalie_name, nation: $nation, position: "GK", age: $age}) MERGE (team:Team {name: $team_name}) MERGE (goalie)-[:PLAYS_FOR]->(team) MERGE (match:Match {id: $match_id}) MERGE (match)-[:HOME_TEAM_GOALIE_STATS]->(goalie) SET goalie.minutes_played = $minutes_played, goalie.sota = $sota, goalie.goals_allowed = $goals_allowed, goalie.saves = $saves, goalie.save_percentage = $save_percentage, goalie.psxg = $psxg, goalie.passes_completed = $passes_completed, goalie.passes_attempted = $passes_attempted, goalie.pass_accuracy = $pass_accuracy, goalie.gk_passes_attempted = $gk_passes_attempted, goalie.throws = $throws, goalie.launch_percentage = $launch_percentage, goalie.launch_average_length = $launch_average_length, goalie.goal_kicks_attempted = $goal_kicks_attempted, goalie.goal_kick_launch_percentage = $goal_kick_launch_percentage, goalie.goal_kick_average_length = $goal_kick_average_length, goalie.crosses_opportunities = $crosses_opportunities, goalie.crosses_stops = $crosses_stops, goalie.crosses_stop_percentage = $crosses_stop_percentage, goalie.opa = $opa, goalie.opa_average_distance = $opa_average_distance

ChatGPT said:

ChatGPT

This code snippet defines a query to insert or update home team goalie stats in a Neo4j database. Here's a breakdown of its purpose and what it does:

Purpose:

To store or update stats for goalkeepers (goalies) in the home team during a match in a graph database, ensuring relationships between players, their teams, and matches are accurately represented.

What It Does:

1. **Create or Match Nodes**:
    a. **Player (goalie)**: A goalkeeper node is created or matched based on their name, nationality, position ("GK"), and age.
    b. **Team (team)**: The team node is created or matched based on the team name.
    c. **Match (match)**: The match node is created or matched based on a unique match ID.
2. **Create Relationships**:
    a. **Player-to-Team**: A PLAYS_FOR relationship is created between the goalie and their team.
    b. **Match-to-Goalie**: A HOME_TEAM_GOALIE_STATS relationship is created between the match and the goalie.
3. **Update Properties**:
    a. The goalie node's properties (like minutes played, saves, and passes completed) are updated with the provided stats using the SET clause.

Required Parameters:

The query relies on the following parameters:

- goalie_name: Name of the goalie.
- nation: Nationality of the goalie.
- age: Age of the goalie.
- team_name: Name of the team the goalie plays for.
- match_id: Unique identifier for the match.
- Various stats (minutes_played, sota, goals_allowed, etc.) representing the goalie's performance during the match.

Example Python Implementation:

If the home_team_goaly_stats is a DataFrame, each row contains the stats for a goalie. The query ensures data integrity by avoiding duplicates using MERGE.

python
CopyEdit

```
for _, goalie_stats in home_team_goaly_stats.iterrows():
    session.run(goalie_query, {
        "goalie_name": goalie_stats[('Unnamed: 0_level_0', 'Player')],
        "nation": goalie_stats[('Unnamed: 1_level_0', 'Nation')],
        "age": goalie_stats[('Unnamed: 2_level_0', 'Age')],
        "match_id": match_id,
        "team_name": home_team_name,
        "minutes_played": goalie_stats[('Unnamed: 3_level_0', 'Min')],
        "sota": goalie_stats[('Shot Stopping', 'SoTA')],
        "goals_allowed": goalie_stats[('Shot Stopping', 'GA')],
        "saves": goalie_stats[('Shot Stopping', 'Saves')],
        "save_percentage": goalie_stats[('Shot Stopping', 'Save%')],
        "psxg": goalie_stats[('Shot Stopping', 'PSxG')],
        "passes_completed": goalie_stats[('Launched', 'Cmp')],
        "passes_attempted": goalie_stats[('Launched', 'Att')],
        "pass_accuracy": goalie_stats[('Launched', 'Cmp%')],
        "gk_passes_attempted": goalie_stats[('Passes', 'Att (GK)')],
        "throws": goalie_stats[('Passes', 'Thr')],
        "launch_percentage": goalie_stats[('Passes', 'Launch%')],
        "launch_average_length": goalie_stats[('Passes', 'AvgLen')],
        "goal_kicks_attempted": goalie_stats[('Goal Kicks', 'Att')],
        "goal_kick_launch_percentage": goalie_stats[('Goal Kicks', 'Launch%')],
        "goal_kick_average_length": goalie_stats[('Goal Kicks', 'AvgLen')],
        "crosses_opportunities": goalie_stats[('Crosses', 'Opp')],
        "crosses_stops": goalie_stats[('Crosses', 'Stp')],
        "crosses_stop_percentage": goalie_stats[('Crosses', 'Stp%')],
        "opa": goalie_stats[('Sweeper', '#OPA')],
        "opa_average_distance": goalie_stats[('Sweeper', 'AvgDist')],
    })
```

Things to Ensure:

1. **Null Values**: Check for missing or NaN values in the DataFrame columns before executing the query. Missing values can cause query failures in Neo4j.
2. **Unique Match IDs**: The match_id should be a unique identifier for each match to avoid overwriting data.
3. **Consistent Data Structure**: Ensure column names and hierarchical levels in the DataFrame match the keys used in the query.

4o

You said:

RUN_ME.ipynb
File
query for how does player x absence affect a teams win rate in away games

ChatGPT said:

ChatGPT

It looks like you've uploaded a Jupyter Notebook file (RUN_ME.ipynb). Do you want me to extract relevant information, modify the notebook, or analyze the data to create the query for player absence and team win rate in away games? Let me know how you'd like to proceed!

4o

You said:

Neo.ClientError.Statement.SyntaxError Expression in WITH must be aliased (use AS) (line 5, column 90 (offset: 300)) "WITH team, match, CASE WHEN p IS NULL THEN "Absent" ELSE "Present" END AS player_status, match.home_score, match.away_score"

ChatGPT said:

ChatGPT

The error indicates that the expression in the WITH clause requires an alias. Let me fix the query by ensuring the CASEexpression is properly aliased with AS. Here's the corrected query:

cypher
CopyEdit
```
MATCH (player:Player {name: $player_name})-[:PLAYS_FOR]->(team:Team)<-[:AWAY_TEAM]-(match:Match)
WITH team, match, player
OPTIONAL MATCH (match)-[:AWAY_TEAM_PLAYER_STATS]->(p:Player)
WHERE p.name = $player_name
WITH team, match,
```

```
    CASE WHEN p IS NULL THEN "Absent" ELSE "Present" END AS player_status,
    match.home_score AS home_score, match.away_score AS away_score
WHERE player_status = "Absent"
WITH team, player_status,
    COUNT(CASE WHEN away_score > home_score THEN 1 END) AS wins,
    COUNT(*) AS total
RETURN team.name AS team_name, player_status, wins, total,
     ROUND(toFloat(wins) / total * 100, 2) AS win_rate

UNION

MATCH (player:Player {name: $player_name})-[:PLAYS_FOR]->(team:Team)<-
[:AWAY_TEAM]-(match:Match)
WITH team, match, player
OPTIONAL MATCH (match)-[:AWAY_TEAM_PLAYER_STATS]->(p:Player)
WHERE p.name = $player_name
WITH team, match,
    CASE WHEN p IS NULL THEN "Absent" ELSE "Present" END AS player_status,
    match.home_score AS home_score, match.away_score AS away_score
WHERE player_status = "Present"
WITH team, player_status,
    COUNT(CASE WHEN away_score > home_score THEN 1 END) AS wins,
    COUNT(*) AS total
RETURN team.name AS team_name, player_status, wins, total,
     ROUND(toFloat(wins) / total * 100, 2) AS win_rate
```

Changes Made:

1. Aliased the CASE expression in the WITH clause with AS player_status.
2. Explicitly aliased match.home_score and match.away_score as home_score and away_score.

This query should now execute correctly in Neo4j. Let me know if you encounter any other issues or need further assistance!

4o

mport altair as alt # Create the grouped bar chart chart = alt.Chart(df).mark_bar().encode(x=alt.X('goals_scored:O', title='Goals Scored', axis=alt.Axis(labelAngle=0)), # Group bars by goals scored y=alt.Y('frequency:Q', title='Frequency'), # Frequency on the y-axis color=alt.Color('team_type:N', # Color by

team type scale=alt.Scale(domain=['home', 'away'], range=['blue', 'red']), legend=alt.Legend(title="Team Type")), column=alt.Column('team_type:N', title=None) # Separate bars for home and away).properties(title='Frequency of Goals Scored: Home vs. Away', width=300, height=400) chart.show()why is my graph not working with driver.session(database="neo4j") as session: query = """ MATCH (m:Match) WHERE m.home_score IS NOT NULL WITH m.home_score AS score, 'home' AS team_type RETURN team_type, score AS goals_scored, COUNT(*) AS frequency ORDER BY goals_scored UNION ALL MATCH (m:Match) WHERE m.away_score IS NOT NULL WITH m.away_score AS score, 'away' AS team_type RETURN team_type, score AS goals_scored, COUNT(*) AS frequency ORDER BY goals_scored """ result = session.run(query) data = [{"team_type": record["team_type"], "goals_scored": record["goals_scored"], "frequency": record["frequency"]} for record in result] data

ChatGPT said:

ChatGPT

The issue could be that the data being passed to alt.Chart(df) is not in the proper format. Altair expects a DataFrame, and you're currently using a list of dictionaries.

You should convert data into a pandas.DataFrame before passing it into the Altair chart. Here's how you can fix it:

Updated Code:

python
CopyEdit
```
import altair as alt
import pandas as pd

# Execute Neo4j query and get data
with driver.session(database="neo4j") as session:
    query = """
    MATCH (m:Match)
    WHERE m.home_score IS NOT NULL
    WITH m.home_score AS score, 'home' AS team_type
    RETURN
      team_type,
      score AS goals_scored,
      COUNT(*) AS frequency
    ORDER BY goals_scored
```

```
  UNION ALL
  MATCH (m:Match)
  WHERE m.away_score IS NOT NULL
  WITH m.away_score AS score, 'away' AS team_type
  RETURN
    team_type,
    score AS goals_scored,
    COUNT(*) AS frequency
  ORDER BY goals_scored
  """
  result = session.run(query)
  data = [{"team_type": record["team_type"],
         "goals_scored": record["goals_scored"],
         "frequency": record["frequency"]} for record in result]

# Convert the data to a pandas DataFrame
df = pd.DataFrame(data)

# Create the grouped bar chart
chart = alt.Chart(df).mark_bar().encode(
  x=alt.X('goals_scored:O', title='Goals Scored', axis=alt.Axis(labelAngle=0)),  # Group
bars by goals scored
  y=alt.Y('frequency:Q', title='Frequency'),      # Frequency on the y-axis
  color=alt.Color('team_type:N',                # Color by team type
            scale=alt.Scale(domain=['home', 'away'], range=['blue', 'red']),
            legend=alt.Legend(title="Team Type")),
  column=alt.Column('team_type:N', title=None)    # Separate bars for home and away
).properties(
  title='Frequency of Goals Scored: Home vs. Away',
  width=300,
  height=400
)

chart.show()
```

Key Changes:

1. **Convert the data to a DataFrame**: df = pd.DataFrame(data)
2. **Pass the DataFrame to Altair**: alt.Chart(df)

Now, the Altair chart should work as expected. The data is structured correctly and passed as a pandas DataFrame, allowing Altair to plot it without issues.

what does this do record = result.single()

ChatGPT said:

ChatGPT

The line record = result.single() in Neo4j's Python driver (neo4j) does the following:

- **result**: This is the result of the query you ran. It is an iterable object that contains one or more records (or rows) of data returned by the Cypher query.
- **single()**: This method returns a **single record** from the result, and it expects that the query will return **at most one record**. If the query returns multiple records, it will raise an exception.

What it does in this context:

- **record = result.single()**:
  - This tries to get the first (and ideally only) record from the result.
  - If the result contains more than one record, it will raise an exception.
  - If no record is found, it will return None.

Example:

If your query is designed to return only one row of data (e.g., the player's win ratio for a specific match), then single() is a convenient way to fetch just that one record.

If the query returns a result (e.g., the player's statistics), record will be a dictionary-like object containing the fields defined in the RETURN clause of the query.

Example with a query result:

Suppose the query returns:

plaintext
CopyEdit

```
+-------------------+-------------------+
| player_name       | playing_win_ratio |
+-------------------+-------------------+
| 'Nick Pope'       | 55.6              |
+-------------------+-------------------+
```

After calling result.single(), you get:

python
CopyEdit
```
record = {
   'player_name': 'Nick Pope',
   'playing_win_ratio': 55.6
}
```

You can then access the values like this:

python
CopyEdit
```
player_name = record['player_name']  # 'Nick Pope'
playing_win_ratio = record['playing_win_ratio']  # 55.6
```

If no result is returned (for example, if no players match the query), record will be None. This is why you can safely check for the presence of a record using an if record: check before accessing its fields.

Summary:

- **single()**: Retrieves a single record from the query result.
    - Returns None if no record is found.
    - Returns a dictionary-like object with the data if a record is found.

MATCH (player:Player {name:$player_name}) MATCH (player)-[:IS_IN_SQUAD]->(squad:Squad) MATCH (team:Team)-[:HAS_SQUAD]->(squad) MATCH (squad)-[:HOME_TEAM|AWAY_TEAM]-(match:Match) MATCH (match)-[:IS_OF_GW]->(game_week:GameWeek) WHERE game_week.season = "2022-2023" WHERE NOT (player)-[:PLAYS_IN]->(match) WITH player.name AS player_name, team.name AS team_name, SUM(CASE WHEN ((match)-[:HOME_TEAM]->(squad) AND match.home_score > match.away_score) OR ((match)-[:AWAY_TEAM]->(squad) AND

match.away_score > match.home_score) THEN 1 ELSE 0 END) AS wins, SUM(CASE WHEN match.home_score = match.away_score THEN 1 ELSE 0 END) AS draws, SUM(CASE WHEN ((match)-[:HOME_TEAM]->(squad) AND match.home_score < match.away_score) OR ((match)-[:AWAY_TEAM]->(squad) AND match.away_score < match.home_score) THEN 1 ELSE 0 END) AS losses WITH player_name, wins, draws, losses, wins + draws + losses AS total_games RETURN player_name, CASE WHEN total_games > 0 THEN (toFloat(wins) / total_games) * 100 ELSE 0 END AS playing_win_ratio

ChatGPT said:

ChatGPT

It seems you're trying to calculate the win ratio for a player when they **don't** participate in a match (i.e., when the player is **not playing** in the match). The query you've written is almost correct, but you might need to adjust the way you're calculating wins, draws, and losses for the matches in which the player doesn't participate.

The current query attempts to do this, but the logic may need some fine-tuning to ensure that the conditions for wins, losses, and draws are handled correctly, particularly in the WHERE NOT (player)-[:PLAYS_IN]->(match) part.

Adjustments to your Query

Here's an adjusted version of your query that should work as expected:

1. Ensuring NOT (player)-[:PLAYS_IN]->(match) correctly identifies matches where the player did not play.
2. Fixing the logic to make sure wins, draws, and losses are counted appropriately when the player doesn't play.
3. Including all relevant conditions for win/loss calculations based on team performance, not the individual player's participation.

cypher
CopyEdit

```
MATCH (player:Player {name: $player_name})
MATCH (player)-[:IS_IN_SQUAD]->(squad:Squad)
MATCH (team:Team)-[:HAS_SQUAD]->(squad)
MATCH (squad)-[:HOME_TEAM|AWAY_TEAM]-(match:Match)
MATCH (match)-[:IS_OF_GW]->(game_week:GameWeek)
WHERE game_week.season = "2022-2023"
AND NOT (player)-[:PLAYS_IN]->(match)  // Filter for matches where player is not
```

```
playing
WITH
  player.name AS player_name,
  team.name AS team_name,
  SUM(CASE
    WHEN (match)-[:HOME_TEAM]->(squad) AND match.home_score >
match.away_score THEN 1
    WHEN (match)-[:AWAY_TEAM]->(squad) AND match.away_score >
match.home_score THEN 1
    ELSE 0 END) AS wins,
  SUM(CASE WHEN match.home_score = match.away_score THEN 1 ELSE 0 END) AS
draws,
  SUM(CASE
    WHEN (match)-[:HOME_TEAM]->(squad) AND match.home_score <
match.away_score THEN 1
    WHEN (match)-[:AWAY_TEAM]->(squad) AND match.away_score <
match.home_score THEN 1
    ELSE 0 END) AS losses
WITH
  player_name,
  wins,
  draws,
  losses,
  wins + draws + losses AS total_games
RETURN
  player_name,
  CASE
    WHEN total_games > 0 THEN (toFloat(wins) / total_games) * 100
    ELSE 0
  END AS not_playing_win_ratio
```

Explanation of Changes:

1. **AND NOT (player)-[:PLAYS_IN]->(match)**: This ensures that we are looking at matches where the player did not play.
2. **The CASE Statements**: I adjusted the CASE statements to properly count wins and losses for the team (not for the player), depending on whether the player's team won or lost the match.
    a. A win is counted if the player's team (either home or away) has a higher score than the opponent.

   b. A loss is counted if the player’s team (either home or away) has a lower score than the opponent.
   c. A draw is counted when the scores are equal.
3. **Return the not_playing_win_ratio**: This calculates the win ratio for the player's team in matches the player did not play.

Outcome:

This query will return the win ratio for the team in the matches the player did not participate in, calculated as:

scss
CopyEdit
(not_playing_win_ratio) = (number of wins) / (total games) * 100

Let me know if you'd like further modifications or clarifications!

this is good but it doesn't count the wins and losses correctly only the draws. this is the old working code I had for calculating wins and losses query = """ MATCH (player:Player {name: $player_name}) MATCH (player)-[:PLAYS_FOR]->(team:Team) MATCH (team)-[:HOME_TEAM|AWAY_TEAM]-(m:Match) MATCH (player)-[stats:HOME_TEAM_PLAYER_STATS|AWAY_TEAM_PLAYER_STATS]-(m) WITH player.name AS player_name, SUM(CASE WHEN (team.name = m.home_team AND m.home_score > m.away_score) OR (team.name = m.away_team AND m.away_score > m.home_score) THEN 1 ELSE 0 END) AS wins, SUM(CASE WHEN m.home_score = m.away_score THEN 1 ELSE 0 END) AS draws, SUM(CASE WHEN (team.name = m.home_team AND m.home_score < m.away_score) OR (team.name = m.away_team AND m.away_score < m.home_score) THEN 1 ELSE 0 END) AS losses WITH player_name, wins, draws, losses, wins + draws + losses AS total_games RETURN player_name, (toFloat(wins) / total_games)*100 AS playing_win_ratio

ChatGPT said:

ChatGPT

Thank you for providing your working version! I see the key part that I missed: the relationship types HOME_TEAM_PLAYER_STATS and AWAY_TEAM_PLAYER_STATS, as well as ensuring we are counting wins, losses, and draws properly.

I'll adjust the query to integrate the correct logic based on your working code, using the proper relationships and conditions for wins and losses, while still filtering for the 2022-2023 season.

Updated Query:

cypher
CopyEdit

```
MATCH (player:Player {name: "Callum Wilson"})
MATCH (player)-[:IS_IN_SQUAD]->(squad:Squad)
MATCH (team:Team)-[:HAS_SQUAD]->(squad)  // Define team
MATCH (player)-[plays:PLAYS_IN]->(match:Match)
MATCH (match)-[:HOME_TEAM]->(home_squad:Squad)
MATCH (match)-[:AWAY_TEAM]->(away_squad:Squad)
MATCH (match)-[:IS_OF_GW]->(game_week:GameWeek)
WHERE game_week.season = "2022-2023"  // Filter for the 2022-2023 season
WITH player.name AS player_name,
   team.name AS team_name,
   home_squad.name AS home_team_name,
   away_squad.name AS away_team_name,
   match.home_score AS home_score,
   match.away_score AS away_score
WITH player_name,
   SUM(CASE WHEN (team_name = home_team_name AND home_score >
away_score) OR
         (team_name = away_team_name AND away_score > home_score) THEN 1
ELSE 0 END) AS wins,
   SUM(CASE WHEN home_score = away_score THEN 1 ELSE 0 END) AS draws,
   SUM(CASE WHEN (team_name = home_team_name AND home_score <
away_score) OR
         (team_name = away_team_name AND away_score < home_score) THEN 1
ELSE 0 END) AS losses
WITH player_name, wins, draws, losses
RETURN player_name, wins, draws, losses
```

Explanation of Adjustments:

1. **Fixed Win and Loss Counting:**
    a. I kept the logic to count wins and losses based on comparing home_score and away_score.

   b. A win is counted when the team's score is higher than the opponent's score for that match (either home or away).
   c. A loss is counted similarly, but in reverse (the team loses if the opponent's score is higher).
2. **Draws:**
   a. Draws are correctly counted when the home_score equals the away_score.
3. **Fixed Filtering for the 2022-2023 Season:**
   a. We're still filtering by the 2022-2023 season using game_week.season = "2022-2023".
4. **Final Return:**
   a. We're returning the player_name, wins, draws, and losses.

This should now correctly count the wins, draws, and losses for Callum Wilson in the 2022-2023 season. Please give this a try, and let me know if you encounter any further issues!

import os from neo4j import GraphDatabase from neo4j.exceptions import ServiceUnavailable from dotenv import load_dotenv from utilities.cypher_query_loader import CypherQueryLoader from utilities.fbref import FBRefDataLoader, FBRefDataFormatter import json import logging logging.basicConfig(level=logging.INFO, format="%(message)s") NEO4J_URI="neo4j+s://14544b1e.databases.neo4j.io" NEO4J_USERNAME='neo4j' NEO4J_PASSWORD='DSB0RkgvGa1atsHc5KoiVPst4l95qg_iMADHrRDXmzA' CQL = CypherQueryLoader(os.path.join(os.getcwd(), 'queries/')) DataLoader = FBRefDataLoader(os.path.join(os.getcwd(),'data/')) DataFormatter = FBRefDataFormatter() driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USERNAME, NEO4J_PASSWORD)) gameweeks_season_query = CQL.load_query("season_gameweeks_creation") player_creation_query = CQL.load_query("player_creation") match_creation_query = CQL.load_query("match_creation") player_gamestats_query = CQL.load_query("player_game_stats") goalie_gamestats_query = CQL.load_query("goalie_game_stats") data_directory = os.path.join(os.getcwd(),'data') for season_directory in DataLoader.listdir(data_directory): print(season_directory) match_reports = DataLoader.load_data("fixtures", season_directory) team_details = DataLoader.load_data("teams", season_directory) player_details = DataLoader.load_data("players", season_directory) player_details = DataFormatter.add_team_name_to_player(player_details, team_details) game_weeks_data = [{"gw_number": f"GW{gw}", "season_name": season_directory} for gw in range(1, 39)] match_data, player_stats_data, goalkeeper_stats_data = DataFormatter.split_match_data(match_reports) # Execute Queries with

driver.session(database="neo4j") as session: session.run(gameweeks_season_query, parameters={"game_weeks": game_weeks_data}) session.run(player_creation_query, parameters={"players": player_details, "season_name": season_directory}) session.run(match_creation_query, parameters={"matches": match_data, "season_name": season_directory}) session.run(player_gamestats_query, parameters={ "player_stats": player_stats_data, "season_name": season_directory }) session.run(goalie_gamestats_query, parameters={ "goalkeeper_stats": goalkeeper_stats_data, "season_name": season_directory }) 150 except OSError as e: --> 151 raise ValueError(f"Cannot resolve address {address}") from e 152 return _resolved_addresses_from_info(info, address._host_name) ValueError: Cannot resolve address 14544b1e.databases.neo4j.io:7687

ChatGPT said:

ChatGPT

The error **"ValueError: Cannot resolve address 14544b1e.databases.neo4j.io:7687"** suggests that your Python script is unable to connect to the Neo4j database at the specified URI. Here are a few possible reasons and solutions:

Possible Causes & Solutions

1. Incorrect or Expired Neo4j URI

- The URI (neo4j+s://14544b1e.databases.neo4j.io) may be incorrect or no longer valid.
- Solution: Verify the database URI by logging into Neo4j Aura and checking the **Connection Details**.

2. Network Issues

- Your internet connection might be down, or your firewall/VPN might be blocking access to Neo4j.
- Solution:
    - Check your internet connection.
    - Try running ping 14544b1e.databases.neo4j.io in the command line to see if the address is reachable.
    - If you're on a corporate or university network, they may have security restrictions—try connecting via a personal network.

3. Incorrect Authentication Credentials

- If the username or password is incorrect, the connection may fail.
- Solution:
    - Double-check the **username** (neo4j) and **password** (DSB0RkgvGa1atsHc5KoiVPst4l95qg_iMADHrRDXmzA).
    - If unsure, reset the password via the Neo4j Aura console.

4. Neo4j Database is Not Running

- If the database has been stopped or suspended due to inactivity, it won't accept connections.
- Solution:
    - Go to your Neo4j Aura dashboard and check if the database is running.
    - If it's paused, restart it and try connecting again.

5. Incorrect Port or Protocol

- Neo4j uses bolt:// (default port 7687) for connections. If the database is configured differently, you might need to adjust your connection string.
- Solution:
    - Try using bolt:// instead of neo4j+s:// in your NEO4J_URI:

python
CopyEdit
```
NEO4J_URI = "bolt://14544b1e.databases.neo4j.io:7687"
```

6. DNS Resolution Issues

- If your system cannot resolve 14544b1e.databases.neo4j.io to an IP address, it could be a DNS issue.
- Solution:
    - Try manually resolving the domain by running:

lua
CopyEdit
```
nslookup 14544b1e.databases.neo4j.io
```

    - If this fails, try changing your DNS settings to Google DNS (8.8.8.8 and 8.8.4.4).

7. SSL/TLS Configuration Issue

- If your Neo4j database does not support secure connections (neo4j+s://), you might need to disable encryption.
- Solution:
    - Try using neo4j:// instead of neo4j+s://:

python
CopyEdit
NEO4J_URI = "neo4j://14544b1e.databases.neo4j.io:7687"

Next Steps

1. **Check if your database is running in the Neo4j console.**
2. **Verify your URI, username, and password.**
3. **Try alternative connection strings (bolt:// or neo4j://).**
4. **Check if your network or firewall is blocking access.**
5. **Run ping or nslookup to see if the address resolves.**

Let me know what you've tried, and we can debug further! 🚀

4o

You said:

driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USERNAME, NEO4J_PASSWORD)),connection_timeout=60,max_connection_lifetime=300)

ChatGPT said:

ChatGPT

It looks like there's a syntax error in your GraphDatabase.driver() call. The issue is that the closing parenthesis for GraphDatabase.driver(...) is in the wrong place, which makes connection_timeout and max_connection_lifetimeappear outside the function call.

Corrected Code

python
CopyEdit

```
driver = GraphDatabase.driver(
   NEO4J_URI,
   auth=(NEO4J_USERNAME, NEO4J_PASSWORD),
   connection_timeout=60,
   max_connection_lifetime=300
)
```

Explanation of Fix

1. The incorrect version:

python
CopyEdit
```
driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USERNAME,
NEO4J_PASSWORD)), connection_timeout=60, max_connection_lifetime=300)
```

   a. The GraphDatabase.driver() function call is closed **too early** (before connection_timeout and max_connection_lifetime).
   b. The connection_timeout=60 and max_connection_lifetime=300 are mistakenly outside the function call.
2. **Fixed version:**
   a. The arguments are properly placed **inside** the GraphDatabase.driver() function.

Let me know if you're still facing issues! 🚀

4o

You said:

how long will it run before tmeoit

ChatGPT said:

ChatGPT

Your connection parameters include:

- connection_timeout=60: This means the driver will wait **up to 60 seconds** when trying to establish a connection before throwing a timeout error.

- max_connection_lifetime=300: This means that each connection in the pool will be **closed and replaced after 300 seconds (5 minutes)** to avoid stale connections.

However, **these parameters do not directly control how long your script will run before timing out**. The total runtime depends on:

1. **Network Issues**: If your database is unreachable (e.g., incorrect URI, firewall blocks, or downtime), the driver will attempt to connect for **60 seconds** before failing.
2. **Query Execution Time**: If a Cypher query takes too long, Neo4j might have its own timeout settings. You can explicitly set timeouts for queries using:

python
CopyEdit
```
session.run("YOUR_QUERY", timeout=120)  # Set a 2-minute query timeout
```

3. **Script Logic**: If your script is processing large data files, it will run as long as needed unless interrupted.

Potential Issue in Your Case

Your error **"Cannot resolve address 14544b1e.databases.neo4j.io:7687"** suggests that the connection isn't being established in the first place. Check:

- If the **Neo4j database is running** and accessible.
- If the **URI is correct** (neo4j+s://... suggests you're using a cloud instance, which should be reachable).
- If **your internet connection/firewall is blocking the request**.

Would you like help debugging the connection issue? 🚀

4o

You said:

---------- KeyError Traceback (most recent call last) Cell In[9], line 44 42 player_details = DataFormatter.add_team_name_to_player(player_details, team_details) 43 game_weeks_data = [{"gw_number": f"GW{gw}", "season_name": season_directory} for gw in range(1, 39)] ---> 44 match_data, player_stats_data, goalkeeper_stats_data = DataFormatter.split_match_data(match_reports) 46 # Execute Queries 47 with driver.session(database="neo4j") as session: File ~/Desktop/project-at2024-

dam/utilities/fbref.py:122, in FBRefDataFormatter.split_match_data(self, match_report) 119 match_data_list.append(match_data) 121 # retrieve the home and away player statistics --> 122 home_player_stats = match["Home_Player_Stats"] 123 away_player_stats = match["Away_Player_Stats"] 125 # for each individual player statistic within the combination of these two lists KeyError: 'Home_Player_Stats'

ChatGPT said:

ChatGPT

The **KeyError: 'Home_Player_Stats'** means that your match_reports data does not contain the expected "Home_Player_Stats" key. This suggests one of the following issues:

Possible Causes & Fixes

1. The Data Structure is Unexpected

Before calling split_match_data(), check what match_reports actually contains. Try printing a sample:

python
CopyEdit
```
print(json.dumps(match_reports[:2], indent=4))  # Print first 2 entries to inspect structure
```

Look for "Home_Player_Stats" inside the JSON structure. If it's missing or named differently, you might need to adjust your code.

2. The Data Loader is Not Parsing JSON Correctly

If match_reports is expected to be a dictionary, check if the data was loaded properly from the source (e.g., a JSON file). If you're using Pandas or another library, make sure it's correctly structured.

3. Some Matches Are Missing Player Stats

If "Home_Player_Stats" is present in some but not all entries, you can handle missing data by adding a fallback:

python

CopyEdit
home_player_stats = match.get("Home_Player_Stats", []) # Default to empty list if missing
away_player_stats = match.get("Away_Player_Stats", [])

This prevents the script from crashing when some matches don't have player stats.

4. Check If the Data Loader Needs Fixing

If the data is being pulled from FBRefDataLoader.load_data(), verify that it's extracting match details correctly. If match_reports is empty or missing fields, the issue may be upstream in how data is fetched.

Would you like to debug this together? 🚀

4o

You said:

SessionExpired: Failed to read from defunct connection IPv4Address(('14544b1e.databases.neo4j.io', 7687)) (ResolvedIPv4Address(('34.78.76.49', 7687)))

ChatGPT said:

ChatGPT

Your **SessionExpired** error indicates that the connection to your Neo4j database has been lost or is defunct. Here are possible causes and solutions:

Possible Causes & Fixes

1. Network Issues (Unstable Internet or Firewall)

- **Check your network stability**

Run a simple ping test in your terminal to see if you can reach Neo4j's domain:

sh
CopyEdit

ping 14544b1e.databases.neo4j.io

If you experience packet loss, your network may be unstable.

- **Try reconnecting with a longer timeout**

Modify your connection settings to allow more time before the connection is considered defunct:

python
CopyEdit
```
driver = GraphDatabase.driver(
   NEO4J_URI,
   auth=(NEO4J_USERNAME, NEO4J_PASSWORD),
   connection_timeout=60,  # Increase timeout
   max_connection_lifetime=600  # Extend lifetime
)
```

2. Neo4j Aura Instance Went Idle

If you are using **Neo4j Aura**, the database might have been paused due to inactivity.
Try restarting it manually from the Neo4j Aura Console.

3. Too Many Open Connections

If your script repeatedly opens sessions without closing them, you might hit a connection limit.
Make sure to properly **close sessions**:

python
CopyEdit
```
with driver.session() as session:
   session.run("MATCH (n) RETURN n LIMIT 1")  # Example query

driver.close()  # Close connection properly
```

4. Using the Wrong Database Name

Ensure you are connecting to the correct database:

python
CopyEdit
```
with driver.session(database="neo4j") as session:  # Replace "neo4j" if needed
    session.run("RETURN 1")
```

If your database name is different, check it in the Neo4j Browser (:use your_database_name).

5. Update the Neo4j Python Driver

If your driver is outdated, upgrade it:

sh
CopyEdit
```
pip install --upgrade neo4j
```

Next Steps

1. Check your internet connection & firewall rules.
2. Restart your Neo4j instance if using Neo4j Aura.
3. Increase timeouts & close sessions properly.
4. Verify you're using the right database name.
5. Update the Neo4j Python driver.

Try these and let me know what happens! 🚀

Cell In[11], line 13 dict = to_insert{'NEO4J_URI':"neo4j+s://14544b1e.databases.neo4j.io", ^ SyntaxError: invalid syntax

ChatGPT said:

ChatGPT

The error arises because the code snippet to_insert{'NEO4J_URI':"neo4j+s://14544b1e.databases.neo4j.io"} is missing a pair of parentheses. The correct syntax for calling a function or method, or working with objects like to_insert, requires parentheses or brackets as appropriate.

If you intended to create a dictionary, but mistakenly used to_insert incorrectly, it should be:

python
CopyEdit
```
to_insert = {'NEO4J_URI': "neo4j+s://14544b1e.databases.neo4j.io"}
```

Alternatively, if to_insert is a function or method, you need parentheses:

python
CopyEdit
```
to_insert({'NEO4J_URI': "neo4j+s://14544b1e.databases.neo4j.io"})
```

Double-check whether to_insert is intended to store the dictionary or is part of a function call. Let me know if you have more context so I can guide you better!

WHAT TO CALL THIS DATABSE PROJECT "Use Case: Predict match outcomes. Query: ""What is the probability of Team A winning against Team B based on recent performance?" Use Case: Analyze key player contributions. Query: ""Which players contribute most to Team X's success against opponents?"" Use Case: Track team performance trends. Query: ""How has Team C's performance changed over the past three seasons compared to other league teams?"" Use Case: Assess the impact of injuries. Query: ""How does Player X's absence affect a Teams win rate in away games?"" Use Case: Explore game factors and scoring patterns. Query: ""What are the common scoring patterns in rainy matches, and which teams adapt best?""""
2/2

ChatGPT said:

ChatGPT

Here are some suggestions for naming your sports analytics database project:

1. **MatchWise** – Emphasizing smart insights into matches and team dynamics.
2. **PlayPredict** – Highlighting the predictive analytics aspect of the project.
3. **TeamPulse** – Reflecting the ability to track team performance trends and key player contributions.
4. **SportIQ** – A sharp, professional name representing intelligence in sports data.
5. **GameLens** – Suggesting a focus on deep analysis of game factors and patterns.

6. **InPlayDB** – A functional yet catchy name for a database with live and historical insights.
7. **WinFactor** – Focusing on uncovering the factors influencing wins and outcomes.
8. **GameDNA** – Symbolizing the decoding of team and match behaviors at a detailed level.
9. **ScoreInsight** – Perfect for exploring scoring patterns and performance factors.
10. **PerformanceLab** – Suggesting an experimental and analytical approach to sports data.

Would you like me to refine these further based on a specific tone (e.g., professional, creative, modern)?

4o

You said:

NO JUST LIKE A TITLE FOR THE REPORT

ChatGPT said:

ChatGPT

Got it! Here are some report title ideas:

1. **"Uncovering Winning Patterns: A Data-Driven Analysis of Team Performance and Player Impact"**
2. **"The Game Decoded: Insights into Match Outcomes, Trends, and Scoring Factors"**
3. **"Predicting Victory: A Comprehensive Study of Team Dynamics and Performance Metrics"**
4. **"From Data to Decisions: Exploring Performance, Injuries, and Match Outcomes in Sports"**
5. **"Scoring Success: An Analytical Approach to Trends, Contributions, and Game Factors"**

Let me know if you want me to tweak the tone!

4o

You said:

We will be using the database application to store data about football games, the players and their teams. This is so that we can predict football match odds. These

predictions will mainly require analysis of match outcomes, individual player performance, and team trends. If possible, we will also take into account weather conditions of where each game is being played.

ChatGPT said:

ChatGPT

Here’s a title tailored to your project description:

"Building the Playbook: A Football Match Prediction Database Integrating Player, Team, and Weather Analytics"

Let me know if you'd like to refine it further!

4o

You said:

no just a general title

ChatGPT said:

ChatGPT

Understood! Here’s a simple, general title:

"Football Match Prediction Database"